

Run Time Polymorphism

Types of polymorphism.

1. Compile time Polymorphism

2. Run time Polymorphism

Compile time Polymorphism	Run time Polymorphism
In Compile time Polymorphism, call is resolved by the compiler .	In Run time Polymorphism, call is not resolved by the compiler.
It is also known as Static binding, Early binding and overloading as well.	It is also known as Dynamic binding, Late binding and overriding as well.
Overloading is compile time polymorphism where more than one methods share the same name with different parameters or signature and different return type.	Overriding is run time polymorphism having same method with same parameters or signature, but associated in a class & its subclass.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers .
It provides fast execution because known early at compile time.	It provides slow execution as compare to early binding because it is known at runtime.
Compile time polymorphism is less flexible as all things execute at compile time.	Run time polymorphism is more flexible as all things execute at run time.

Run Time Polymorphism

1. Selecting an appropriate member function while the program is running is known as run time polymorphism.
2. C++ supports a mechanism known as virtual function to achieve runtime polymorphism

It is a mechanism in which the method called by an object gets associated by name in runtime. Late binding happens when **virtual** keyword is used in member function declaration.

Function Overriding

If we inherit a class into the derived class and provide a definition for one of the base class's function again inside the derived class, then that function is said to be **overridden**, and this mechanism is called **Function Overriding**.

Requirements for Overriding

1. Inheritance should be there. Function overriding cannot be done within a class.
2. Function that is redefined must have exactly the same declaration in both base and derived class, that means same name, same return type and same parameter list.

Binding

Binding is the process which is used by the compiler to convert identifiers (such as variable and function names) into machine language addresses.

Binding types:

Early Binding: When it is done before the program is run, its called **Early Binding** or **Static Binding** or **Compile-time Binding**.

Late Binding: is also known as Dynamic Binding or Runtime binding. Sometimes compiler can't know which function will be called till program is executed (runtime). This is known as late binding.

Function Call Binding with class Objects

```
class Base
{
public:
    void show()
    {
        cout << "Base class\t";
    }
};

class Derived :public Base
{
public:
    void show()
    {
        cout << "Derived Class";
    }
};

int main()
{
    Base b;           //Base class object
    Derived d;        //Derived class object
    b.show();         //Early Binding Occurs
    d.show();
    system("pause");
    return 0;
}
```

/*In the example, we are calling the override function using Base class and Derived class object.

Base class object will call base version of the function and

derived class's object will call the derived version of the function.*/

Function Call Binding using Base class Pointer

```
class Base
{
public:
    void show()
    {
        cout << "Base class\t";
    }
};
class Derived :public Base
{
public:
    void show()
    {
        cout << "Derived Class";
    }
};
int main()
{
    Base *b;           //Base class object
    Derived d;          //Derived class object
    b = &d;
    b->show();
    system("pause");
    return 0;
}
```

although, the object is of Derived class, still Base class's method is called. This happens due to Early Binding.

Compiler on seeing **Base class's pointer**, set call to Base class's **show()** function, without knowing the actual object type.

Virtual Functions

- Virtual Function is a function in base class, which is overridden in the derived class, and which tells the compiler to perform **Late Binding** on this function.
- Virtual Keyword is used to make a member function of the base class Virtual.

Rules for Virtual Functions

1. The virtual functions must be members of some class.
2. They cannot be static members
3. They are accessed by using object pointers.
4. A virtual function can be a friend of another class.
5. A virtual function in a base class must be defined, even though it may not be used.
6. The prototypes of the base class version of a virtual function and all the derived class versions must be identical. If two functions with the same name have different prototypes, C++ considers them as overloaded functions and the virtual function mechanism is ignored.

7. We cannot have virtual constructors ,but we can have virtual destructors.

8. While a base pointer can point to any type of the derived object, the reverse is not true. That is to say, we cannot use a pointer to a derived class to access an object of the base type.

9. When a base pointer points to a derived class, incrementing or decrementing it will not make it to point to the next object of the derived class.

10. If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class. In such cases, calls will invoke the base function.

Examples of Virtual Functions

```
/ **VIRTUAL FUNCTION** /
```

```
#include<iostream>
#include<string>
using namespace std;
/* ... */
class media
{
protected:
    string title;
    float price;
public:
    media(string s, float a)
    {
        title = s;
        price = a;
    }
    virtual void display()
    {
        }
};
```

//empty virtual function

```
class book : public media
{
    int pages;
public:
    book(string h, float a, int p) :media(h, a)
    {
        pages = a;
    }
    void display();
};
class tape :public media
{
    float time;
public:
    tape(string s, float p, float t) :media(s, p)
    {
        time = t;
    }
    void display();
};
... ..
```

```
|void book::display()  
{  
    cout << "Title:" << title << endl;  
    cout << "Pages:" << pages << endl;  
    cout << "Price:" << price << endl;  
}  
|void tape::display()  
{  
    cout << "Title:" << title << endl;  
    cout << "play time:" << time << endl;  
    cout << "Price:" << price;  
}
```

```

int main()
{
    string title;
    float price, time;
    int pages;
    cout << "ENTER BOOK DETAILS:" << endl;
    cout << "Title:" << endl;
    cin >> title;
    cout << "Price:" << endl;
    cin >> price;
    cout << "Pages:" << endl;
    cin >> pages;
    book book1(title, price, pages);
    cout << "ENTER TAPE DETAILS:" << endl;
    cout << "Title:" << endl;
    cin >> title;
    cout << "Price:" << endl;
    cin >> price;
    cout << "paly Time:" << endl;
    cin >> time;
    tape tape1(title, price, time);
    media *list[2];

```

```

list[0] = &book1;
list[1] = &tape1;
cout << "MEDIA DETAILS:" << endl;
cout << ".....BOOK....." << endl;
list[0]->display();//display book details
cout << ".....TAPE....." << endl;
list[1]->display();//display tape details
system("pause");
return 0;
}

```

ENTER BOOK DETAILS:

Title:

cprogramming

Price:

256.36

Pages:

258

ENTER TAPE DETAILS:

Title:

Cprogramming

Price:

256.8

play Time:

89

MEDIA DETAILS:

.....BOOK.....

Title:cprogramming

Pages:256

Price:256.36

.....TAPE.....

Title:Cprogramming

play time:89

Price:256.8Press any key to continue . . .

Using Virtual Keyword and Accessing Private Method of Derived class

```
#include<iostream>
#include<string>
using namespace std;

{
public:
virtual void show()
{
    cout << "Base class\n";
}
};

class B : public A
{
private:
virtual void show()
{
    cout << "Derived class\n";
}
};

int main()
{
    A *a;
    B b;
    a = &b;
    a->show();
}
```



```
#include<iostream>
#include<string>
using namespace std;
class Base
{
public:
    void display()
    {
        cout << "/n display Base" << endl;
    }
    virtual void show()
    {
        cout << "show Base" << endl;
    }
};
```

```
class Derived :public Base
{
public:
    void display()
    {
        cout << "/n display Derived" << endl;
    }
    virtual void show()
    {
        cout << "show Derived" << endl;
    }
};
```

```
|int main()
{
    Base B;
    Derived D;
    Base *bptr;
    cout << "bptr points to Base" << endl;
    bptr = &B;
    bptr->display();//calls Base Version
    bptr->show(); //calls Base Version
    cout << "bptr points to Derived" << endl;
    bptr = &D;
    bptr->display();//calls Base Version
    bptr->show();//calls Derived Version
    system("pause");
    return 0;
}
```

```
bptra points to Base  
/n display Base  
show Base  
bptra points to Derived  
/n display Base  
show Derived  
Press any key to continue . . . _
```

Pure Virtual Function

Pure virtual Functions are virtual functions with no definition.

They start with **virtual** keyword and ends with = 0.

the syntax for a pure virtual function,

virtual void f() = 0;

Abstract Class

Abstract Class is a class which contains atleast one Pure Virtual function in it.

Abstract classes are used to provide an Interface for its sub classes.

Classes inheriting an Abstract Class must provide definition to the pure virtual function, otherwise they will also become abstract class.

Characteristics of Abstract Class

1. Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
2. Abstract class can have normal functions and variables along with a pure virtual function.
3. Abstract classes are mainly used for Upcasting, so that its derived classes can use its interface.
4. Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.

Example :- Abstract Class

```
#include<iostream>
#include<string>
using namespace std;

/* ... */
/* ... */

class Base           //Abstract base class
{
public:
    virtual void show() = 0;           //Pure Virtual Function
};

class Derived :public Base
{
public:
    void show()
    {
        cout << "Implementation of Virtual Function in Derived class";
    }
};
```

```
int main()
{
    Base obj;           //Compile Time Error
    Base *b;
    Derived d;
    b = &d;
    b->show();
}
```

Virtual Destructors

1. Destructors of the Base class must be made virtual for proper destruction of the object when the program exits.
2. Constructors are never Virtual, only Destructors can be Virtual.

Upcasting without Virtual Destructor

```
class Base
{
public:
    ~Base() { cout << "Base Destructor\t"; }
};

class Derived :public Base
{
public:
    ~Derived() { cout << "Derived Destructor"; }
};

int main()
{
    Base* b = new Derived;    //Upcasting
    delete b;
    system("pause");
    return 0;
}
```

Output :
Base Destructor

Note: **delete b** will only call the Base class destructor, which is undesirable because, then the object of Derived class remains undestructed, because its destructor is never called. Which results in memory leak.

Upcasting with Virtual Destructor

```
class Base
{
public:
    virtual ~Base()
    {
        cout << "Base Destructor\t"<<endl; }
};

class Derived :public Base
{
public:
    ~Derived()
    {
        cout << "Derived Destructor"<<endl; }
};

int main()
{
    Base* b = new Derived;    //Upcasting
    delete b;
    system("pause");
    return 0;
}
```

Output :
Derived Destructor
Base Destructor

Note: When we have Virtual destructor inside the base class, then first Derived class's destructor is called and then Base class's destructor is called, which is the desired behaviour.

Pure Virtual Destructors

1. Pure virtual Destructors are legal in C++. Also, pure virtual Destructors must be defined, which is against the pure virtual behaviour.
2. The only difference between Virtual and Pure Virtual Destructor is, that pure virtual destructor will make its Base class Abstract, hence you cannot create object of that class.
3. There is no requirement of implementing pure virtual destructors in the derived classes.

Pure Virtual Destructors

1. Pure virtual Destructors are legal in C++. Also, pure virtual Destructors must be defined, which is against the pure virtual behaviour.
2. The only difference between Virtual and Pure Virtual Destructor is, that pure virtual destructor will make its Base class Abstract, hence you cannot create object of that class.
3. There is no requirement of implementing pure virtual destructors in the derived classes.

```
|class Base
|{
|public:
|    virtual ~Base() = 0;    //Pure Virtual Destructor
|};
```

```
Base::~~Base() { cout << "Base Destructor"; } //Definition of Pure Virtual Destructor
```

```
|class Derived :public Base
|{
|public:
|    ~Derived() { cout << "Derived Destructor"; }
|};
```
